



Atividade Prática Orientada

Unidade 1 — Back-End e Front-End

Construção de uma aplicação CRUD Full Stack



1. Identificação da Atividade

Item	Descrição
Disciplina	Desenvolvimento de Software (Back-End/Front-End)
Unidade	1 — Back-End e Front-End
Tipo	Atividade Prática Orientada (APO)
Carga horária	10 horas
Modalidade	Individual ou dupla
Entrega	Repositório público no GitHub + vídeo curto (até 5 min)



2. Contextualização

Você foi contratado(a) por uma **startup de gestão de tarefas** para desenvolver o **MVP (Produto Mínimo Viável)** de uma aplicação web de lista de tarefas colaborativas. O sistema precisa permitir que usuários cadastrem, listem, editem e removam tarefas. O Back-End deve expor uma API RESTful e o Front-End deve consumir essa API de forma assíncrona, sem recarregar a página.

Por que esta atividade existe?

Para que você experimente, de forma integrada, o ciclo completo de uma aplicação Full Stack: modelagem, rotas, requisições assíncronas, tratamento de respostas e boas práticas de organização.



3. Objetivos de Aprendizagem

Ao concluir esta atividade, o estudante deverá ser capaz de:

- **Compreender** o fluxo de comunicação entre cliente e servidor via HTTP;
- **Implementar** um servidor Back-End com Node.js e Express, expondo endpoints REST;
- **Desenvolver** uma interface Front-End responsiva com HTML, CSS e JavaScript;
- **Aplicar** requisições assíncronas com a API `fetch()` para consumir a API;
- **Executar** as quatro operações CRUD (Create, Read, Update, Delete);
- **Analisar** erros comuns de integração e propor soluções;
- **Avaliar** a qualidade do código segundo boas práticas de organização.

Taxonomia de Bloom: Aplicar, Analisar e Avaliar.



4. Competências e Habilidades

Competência	Habilidade
Desenvolver aplicações web Full Stack	Implementar Front-End e Back-End integrados
Projetar APIs RESTful	Definir rotas, métodos HTTP e contratos JSON
Programar em JavaScript	Utilizar funções assíncronas, Promises e tratamento de erros
Aplicar boas práticas	Organizar pastas, nomear arquivos, validar dados
Resolver problemas	Decompor a aplicação em etapas menores e gerenciáveis



5. Problema Proposto

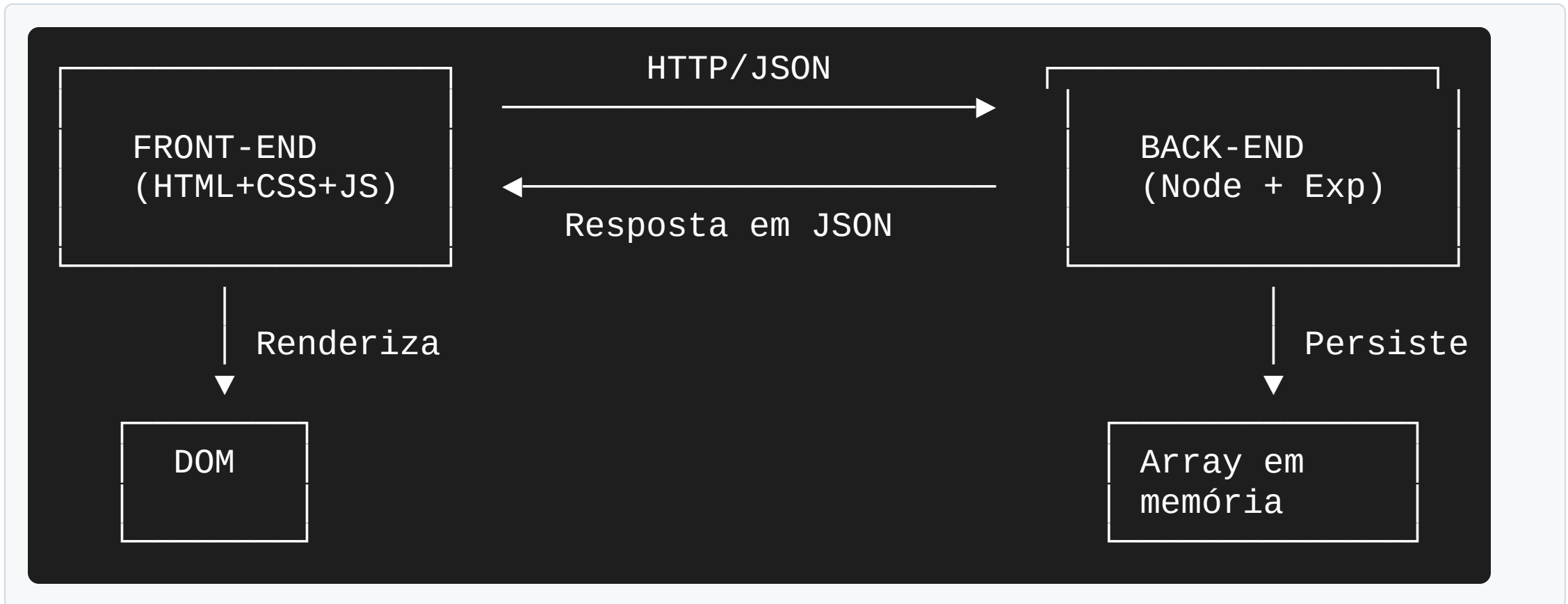
A startup **TaskFlow** precisa de uma aplicação web onde os usuários possam:

1. **Cadastrar** uma nova tarefa informando título, descrição e prioridade;
2. **Listar** todas as tarefas em uma interface limpa e responsiva;
3. **Editar** uma tarefa existente, alterando seus dados;
4. **Excluir** tarefas que já foram concluídas;
5. **Filtrar** tarefas por status (todas, pendentes, concluídas).

A persistência dos dados deve acontecer **em memória** (um array no Back-End), pois o objetivo é didático, não produtivo.



6. Arquitetura da Solução



Fluxo de uma requisição:



7. Tecnologias e Ferramentas

Camada	Tecnologia	Justificativa
Front-End	HTML5 + CSS3	Estrutura e estilo da página
Front-End	JavaScript (ES6+)	Lógica de interação e requisições
Back-End	Node.js	Ambiente de execução JavaScript no servidor
Back-End	Express.js	Framework minimalista para APIs REST
Versionamento	Git + GitHub	Controle de versão e portfólio
Teste	Postman ou Insomnia	Testar endpoints antes de integrar



8. Requisitos Funcionais

Código	Requisito	Prioridade
RF01	Cadastrar tarefa (POST <code>/api/tarefas</code>)	Alta
RF02	Listar todas as tarefas (GET <code>/api/tarefas</code>)	Alta
RF03	Atualizar uma tarefa (PUT <code>/api/tarefas/:id</code>)	Alta
RF04	Excluir uma tarefa (DELETE <code>/api/tarefas/:id</code>)	Alta
RF05	Filtrar tarefas por status	Média
RF06	Marcar tarefa como concluída	Média
RF07	Validar campos obrigatórios	Alta
RF08	Interface responsiva (mobile first)	Média



9. Requisitos Não Funcionais

Código	Requisito
RNF01	O Back-End deve responder em menos de 200 ms para listas pequenas
RNF02	O código deve estar organizado em pastas (<code>public/</code> , <code>src/</code> , <code>routes/</code>)
RNF03	As rotas devem seguir o padrão RESTful
RNF04	O README.md deve conter instruções de execução
RNF05	O projeto deve funcionar localmente sem erros no console



10. Estrutura do Projeto

```
taskflow/  
├── backend/  
│   ├── src/  
│   │   ├── server.js  
│   │   ├── routes/  
│   │   │   └── tarefas.js  
│   │   └── data/  
│   │       └── tarefasStore.js  
│   ├── package.json  
│   └── package-lock.json  
├── frontend/  
│   ├── public/  
│   │   ├── index.html  
│   │   ├── css/  
│   │   │   └── style.css  
│   │   └── js/  
│   │       └── app.js  
├── README.md  
└── .gitignore
```





11. Etapas da Atividade

Etapa 1 — Configuração do Ambiente (1h)

- Instalar Node.js (versão LTS);
- Criar o repositório no GitHub;
- Inicializar o projeto com `npm init -y`;
- Instalar o Express: `npm install express`;
- Instalar o `nodemon` como dependência de desenvolvimento: `npm install --save-dev nodemon`;
- Configurar o `.gitignore` para ignorar `node_modules/`.



Etapa 2 — Back-End: API REST (2h)

Criar o arquivo `backend/src/server.js`:

```
const express = require('express');
const app = express();
const tarefasRouter = require('./routes/tarefas');

app.use(express.json());
app.use('/api/tarefas', tarefasRouter);

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

Criar `backend/src/data/tarefasStore.js`:

```
let tarefas = [];
let nextId = 1;
```





Etapa 2 — Continuação: Rotas REST

Criar `backend/src/routes/tarefas.js`:

```
const express = require('express');
const router = express.Router();
const store = require('../data/tarefasStore');

router.get('/', (req, res) => {
  res.json(store.listar());
});

router.get('/:id', (req, res) => {
  const tarefa = store.buscarPorId(parseInt(req.params.id));
  if (!tarefa) return res.status(404).json({ erro: 'Não encontrada' });
  res.json(tarefa);
});

router.post('/', (req, res) => {
  const { titulo, descricao, prioridade } = req.body;
  if (!titulo) return res.status(400).json({ erro: 'Título obrigatório' });
  const tarefa = store.criar({ titulo, descricao, prioridade });
  res.status(201).json(tarefa);
});

router.put('/:id', (req, res) => {
  const tarefa = store.atualizar(parseInt(req.params.id), req.body);
  if (!tarefa) return res.status(404).json({ erro: 'Não encontrada' });
  res.json(tarefa);
});

router.delete('/:id', (req, res) => {
  const tarefa = store.remover(parseInt(req.params.id));
  if (!tarefa) return res.status(404).json({ erro: 'Não encontrada' });
  res.status(204).send();
});
```





Etapa 2 — Teste dos Endpoints

Antes de prosseguir, teste cada endpoint usando Postman ou `curl`:

```
# Listar tarefas (vazio)
curl http://localhost:3000/api/tarefas

# Criar tarefa
curl -X POST http://localhost:3000/api/tarefas \
  -H "Content-Type: application/json" \
  -d '{"titulo":"Estudar Node","descricao":"Ler docs","prioridade":"alta"}'

# Atualizar
curl -X PUT http://localhost:3000/api/tarefas/1 \
  -H "Content-Type: application/json" \
  -d '{"concluida":true}'

# Remover
curl -X DELETE http://localhost:3000/api/tarefas/1
```



Etapa 3 — Front-End: Estrutura (2h)

Criar `frontend/public/index.html`:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TaskFlow – Lista de Tarefas</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <h1>TaskFlow</h1>
    <p>Sua lista de tarefas colaborativa</p>
  </header>

  <main>
    <section class="form-section">
      <h2>Nova tarefa</h2>
      <form id="form-tarefa">
        <input type="hidden" id="tarefa-id">
        <label>Título*</label>
        <input type="text" id="titulo" required>
        <label>Descrição</label>
        <textarea id="descricao"></textarea>
        <label>Prioridade</label>
        <select id="prioridade">
          <option value="baixa">Baixa</option>
          <option value="media" selected>Média</option>
          <option value="alta">Alta</option>
        </select>
        <button type="submit">Salvar</button>
        <button type="button" id="cancelar">Cancelar</button>
      </form>
    </section>

    <section class="lista-section">
      <h2>Minhas tarefas</h2>
      <div class="filtros">
        <button data-filtro="todas" class="filtro ativo">Todas</button>
        <button data-filtro="pendentes" class="filtro">Pendentes</button>
        <button data-filtro="concluidas" class="filtro">Concluídas</button>
      </div>
      <ul id="lista-tarefas"></ul>
    </section>
  </main>

  <script src="js/app.js"></script>
</body>
```





Etapa 3 — Continuação: CSS Responsivo

Criar `frontend/public/css/style.css`:

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
  font-family: 'Century Gothic', sans-serif;
  background: #f4f4f4;
  color: #272425;
  line-height: 1.6;
}
header {
  background: #C22820;
  color: #fff;
  padding: 24px;
  text-align: center;
}
main {
  max-width: 800px;
  margin: 24px auto;
  padding: 16px;
  display: grid;
  gap: 24px;
  grid-template-columns: 1fr;
}
@media (min-width: 768px) {
  main { grid-template-columns: 1fr 2fr; }
}
.form-section, .lista-section {
  background: #fff;
  padding: 16px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
form label { display: block; margin: 8px 0 4px; font-weight: bold; }
form input, form textarea, form select {
  width: 100%; padding: 8px; border: 1px solid #5C2034;
  border-radius: 4px; font-size: 16px;
}
form button {
  margin-top: 12px; padding: 10px 20px; background: #C22820;
  color: #fff; border: none; border-radius: 4px; cursor: pointer;
}
form button:hover { background: #5C2034; }
.filtros button {
  margin: 4px; padding: 6px 12px; background: #fff;
  border: 2px solid #C22820; color: #C22820;
  border-radius: 4px; cursor: pointer;
}
.filtros button.ativo { background: #C22820; color: #fff; }
#lista-tarefas { list-style: none; margin-top: 16px; }
#lista-tarefas li {
  background: #fff; padding: 12px; margin: 8px 0;
  border-left: 4px solid #C22820; border-radius: 4px;
  display: flex; justify-content: space-between; align-items: center;
}
#lista-tarefas li.concluida { opacity: 0.6; border-left-color: #5C2034; }
```





Criar `frontend/public/js/app.js`:





Etapa 4 — CORS e Integração (1h)

Durante o teste no navegador, você provavelmente receberá o erro:

```
Access to fetch at 'http://localhost:3000/api/tarefas'
from origin 'null' has been blocked by CORS policy
```

Solução: instalar o middleware `cors` no Back-End:

```
npm install cors
```

E adicionar no `server.js`:

```
const cors = require('cors');
app.use(cors());
```





Etapa 5 — Servir o Front-End pelo Back-End (Opcional, +30 min)

Para simplificar, você pode fazer o Back-End servir também os arquivos estáticos do Front-End:

```
const path = require('path');  
app.use(express.static(path.join(__dirname, '../frontend/public')));
```

Agora, ao acessar `http://localhost:3000`, o Back-End entrega o `index.html` e resolve o problema de CORS de uma vez.



Etapa 6 — Documentação e Vídeo (2h)

Criar o **README.md** na raiz do repositório:

```
# TaskFlow — Lista de Tarefas Full Stack

Aplicação CRUD completa integrando Front-End (HTML, CSS, JS) e Back-End (Node.js + Express).

## Tecnologias
HTML5, CSS3, JavaScript (ES6+)
Node.js, Express.js, CORS

## Como executar

### 1. Clonar o repositório
```bash
git clone https://github.com/seu-usuario/taskflow.git
cd taskflow
```

### 2. Instalar dependências do Back-End
```bash
cd backend
npm install
```

### 3. Iniciar o servidor
```bash
npm run dev
```

O servidor estará disponível em `http://localhost:3000`.

## Endpoints da API
Método	Rota	Descrição
GET	/api/tarefas	Lista todas as tarefas
GET	/api/tarefas/:id	Busca tarefa por ID
POST	/api/tarefas	Cria nova tarefa
PUT	/api/tarefas/:id	Atualiza tarefa
DELETE	/api/tarefas/:id	Remove tarefa

## Autoria
```





12. Critérios de Avaliação

| Critério | Peso | Descrição |
|-----------------------|------|---|
| Funcionalidade | 40% | Todos os requisitos funcionais implementados |
| Integração | 20% | Front-End e Back-End se comunicam corretamente |
| Organização do código | 15% | Estrutura de pastas, nomes claros, separação de responsabilidades |
| Boas práticas | 10% | Validação, tratamento de erros, uso correto de verbos HTTP |
| Documentação | 10% | README completo e vídeo demonstrativo |
| Versionamento | 5% | Commits descritivos no GitHub |
| Total | 100% | |



13. Erros Comuns

✗ Erro 1: Não tratar CORS

Sintoma: O Front-End não consegue acessar a API.

Causa: O navegador bloqueia requisições entre origens diferentes.

Solução: Instalar e configurar o middleware `cors()`.

✗ Erro 2: Esquecer de converter o ID

Sintoma: `req.params.id` é uma string, mas o array espera um número.

Causa: O JavaScript não faz coerção automática em comparações estritas.

Solução: Usar `parseInt(req.params.id)`.

✗ Erro 3: Não enviar Content-Type

Sintoma: O Back-End recebe `req.headers` como `undefined`.



14. Boas Práticas

✓ Separação de Responsabilidades

Cada arquivo tem uma única responsabilidade. O `server.js` configura o servidor; o `routes/tarefas.js` define as rotas; o `data/tarefasStore.js` gerencia a persistência. Quando precisar corrigir um bug, você sabe exatamente onde procurar.

✓ Validação no Back-End

Nunca confie no cliente. O Back-End deve validar todos os dados recebidos, mesmo que o Front-End também valide. Isso é uma questão de segurança.

✓ Códigos de Status HTTP Apropriados

| Código | Significado | Quando usar |
|--------|-------------|-----------------------------------|
| 200 | OK | Operação bem-sucedida com retorno |





15. Conexão com as Próximas Unidades

| Unidade | Conexão com a APO |
|-------------------------------|---|
| Unidade 2 — Frameworks | O Back-End atual mistura configuração, rotas e persistência. Com Express, vamos refatorar usando o padrão MVC (Model-View-Controller) e introduzir middlewares para validação e autenticação. |
| Unidade 3 — Web Mobile e PWA | A interface atual funciona no celular, mas não é instalável. Vamos transformá-la em uma PWA com manifest.json e Service Worker para cache e funcionamento offline. |
| Unidade 4 — Framework Próprio | Vamos entender, na prática, como o Express funciona "por dentro", construindo um mini framework com Router, Middleware e Pipeline. |



16. Checklist de Entrega

Antes de submeter, verifique:

- ☐ O repositório no GitHub está público
- ☐ O `README.md` está completo, com instruções de execução
- ☐ O `.gitignore` ignora `node_modules/`
- ☐ O Back-End inicia sem erros (`npm run dev`)
- ☐ Os 5 endpoints REST funcionam (testados no Postman/Insomnia)
- ☐ O Front-End lista, cria, edita, exclui e filtra tarefas
- ☐ O console do navegador não apresenta erros
- ☐ O CSS é responsivo (testado em viewport mobile)
- ☐ O vídeo de até 5 minutos foi gravado e está acessível (YouTube, Drive ou Loom)
- ☐ Os commits são descritivos e incrementais

17. Síntese

Conceitos-chave desta atividade

1. **Front-End e Back-End se comunicam via HTTP**, usando verbos como GET, POST, PUT e DELETE.
2. **APIs RESTful** expõem endpoints que retornam dados em JSON.
3. **fetch()** é a API nativa do navegador para requisições assíncronas.
4. **async/await** torna o código assíncrono mais legível.
5. **CORS** é um mecanismo de segurança do navegador que precisa ser configurado em APIs públicas.
6. **Separação de responsabilidades** é fundamental: cada arquivo, uma função.

Pergunta reflexiva

Você conseguiria explicar, para um colega que não conhece o projeto, **por que** o Front-End e